

LAPORAN MODUL 6 PRAKTIKUM STRUKTUR DATA



Disusun oleh :

Nama : Irvan Cahya Nugraha

NIM : 245410034

Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

LAPORAN STRUKTUR DATA

MODUL 6

PEMANFAATAN ARRAY/LARIK SEBAGAI STACK (TUMPUKAN) DAN QUEUE (ANTRIAN)

A. TUJUAN PRAKTIKUM

Mahasiswa dapat mengimplementasikan tumpukan dan antrian untuk berbagai keperluan dengan menggunakan bahasa pemrograman Java

B. DASAR TEORI

Stack (Tumpukan)

Tumpukan atau stack adalah teknik peletakan data dimana seolah-olah data yang satu diletakkan 'diatas' data yang lain. Dalam tumpukan semua penyisipan dan penghapusan data dilakukan hanya melalui satu pintu yang disebut top (puncak) tumpukan. Tumpukan dapat dibayangkan seperti tumpukan buku dimana hanya buku teratas yang dapat diperoleh kembali dengan satu langkah. Buku-buku yang terdapat dibawah hanya dapat diambil setelah buku yang berada diatasnya diambil (dikeluarkan) terlebih dahulu. Dengan bentuk yang demikian tumpukan dapat dikatakan sebagai struktur data yang bersifat **LIFO (Last In First Out)**.

Dalam tumpukan terdapat 2 buah operasi data yaitu (1) operasi **push** dan (2) operasi **pop**. Operasi **push** adalah operasi untuk menambahkan sebuah data ke dalam tumpukan, sedangkan operasi **pop** adalah operasi untuk mengambil (menghapus) data dari dalam tumpukan.

Queue (Antrian)

Antrian atau Queue adalah teknik peletakan data dimana seolah-olah data yang satu diletakkan 'dibelakang' data yang lain. Dalam antrian semua penyisipan dan penghapusan data dilakukan melalui dua pintu yaitu belakang dan depan. Pintu belakang digunakan sebagai jalan masuk data, sedangkan pintu depan digunakan sebagai jalan keluar data.

Antrian dapat dibayangkan seperti antrian para perima zakat. Orang yang ingin mendapatkan zakat harus mengantri dengan cara masuk melalui sisi belakang. Apabila seorang penerima telah terlayani maka dia harus meninggalkan antrian melalui sisi depan.

Dengan bentuk yang demikian antrian dapat dikatakan sebagai struktur data yang bersifat **FIFO (First In First Out)** artinya pengantri yang datang paling awal akan keluar paling dahulu.

Dalam antrian terdapat 2 buah operasi data yaitu (1) operasi **enQueue** dan (2) operasi **deQueue**. Operasi **enQueue** adalah operasi untuk menambahkan sebuah data ke dalam antrian dan dilakukan pada pintu belakang, sedangkan operasi **deQueue** adalah operasi untuk mengambil (menghapus) data dari dalam antrian dan dilakukan pada pintu depan.

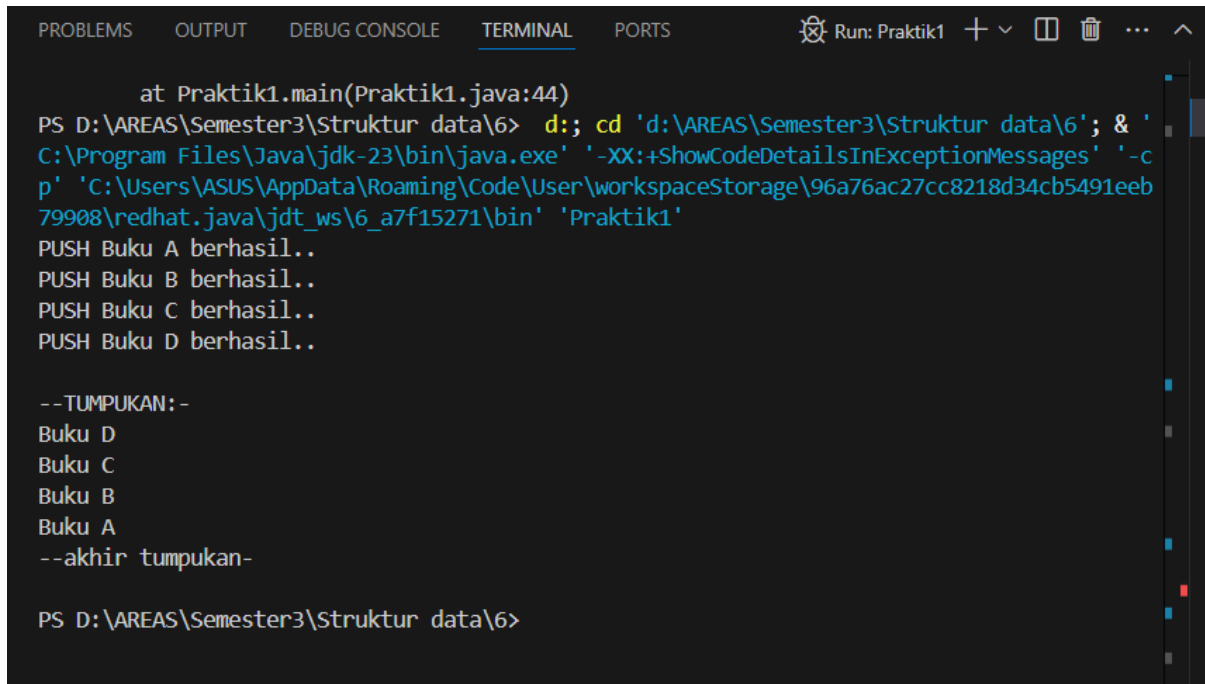
Dalam implementasinya, sebuah antrian pasti memiliki kapasitas atau batas daya tampung antrian. Dalam ilustrasi di atas, kapasitas antrian biasanya disesuaikan dengan banyaknya zakat yang diberikan. Apabila hanya tersedia zakat untuk 10 orang maka kapasitas antrian hanya akan menampung 10 orang, sehingga orang ke-11, ke-12, dan seterusnya pasti akan ditolak untuk masuk ke dalam antrian.

C. PEMBAHASAN LISTING

PRAKTIK

PRAKTIK

Praktik 1



```
at Praktik1.main(Praktik1.java:44)
PS D:\AREAS\Semester3\Struktur data\6> d:; cd 'd:\AREAS\Semester3\Struktur data\6'; & '
C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-c
p' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491eeb
79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik1'
PUSH Buku A berhasil..
PUSH Buku B berhasil..
PUSH Buku C berhasil..
PUSH Buku D berhasil..

--TUMPUKAN:-
Buku D
Buku C
Buku B
Buku A
--akhir tumpukan-

PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik1 {
    public static int N = 5;
    public static int atas = -1;

    public static void PUSH(String tumpukan[], String data
) {
        if (atas == N - 1) { //jika tumpukan penuh
            System.out.println(
"maap, tumpukan penuh, PUSH " + data +
" tidak dapat dilakukan");
        } else { //jika tumpukan tidak penuh
            atas = atas + 1;
            tumpukan[atas] = data;
            System.out.println("PUSH " + data +
" berhasil..");
        }
    }

    public static String POP(String tumpukan[]) {
        String hasil;
        if (atas < 0) { //jika tumpukan kosong
            hasil = "TUMPUKAN KOSONG, POP GAGAL DILAKUKAN"
;
        } else { //jika tumpukan tidak kosong
            hasil = tumpukan[atas];
            atas = atas - 1;
        }
        return (hasil);
    }

    public static void lihatTumpukan(String tumpukan[]) {
        System.out.println("");
        System.out.println("--TUMPUKAN:-");
        for (int i = atas; i >= 0; i--) {
            System.out.println(tumpukan[i]);
        }
        System.out.println("--akhir tumpukan-");
        System.out.println("");
    }

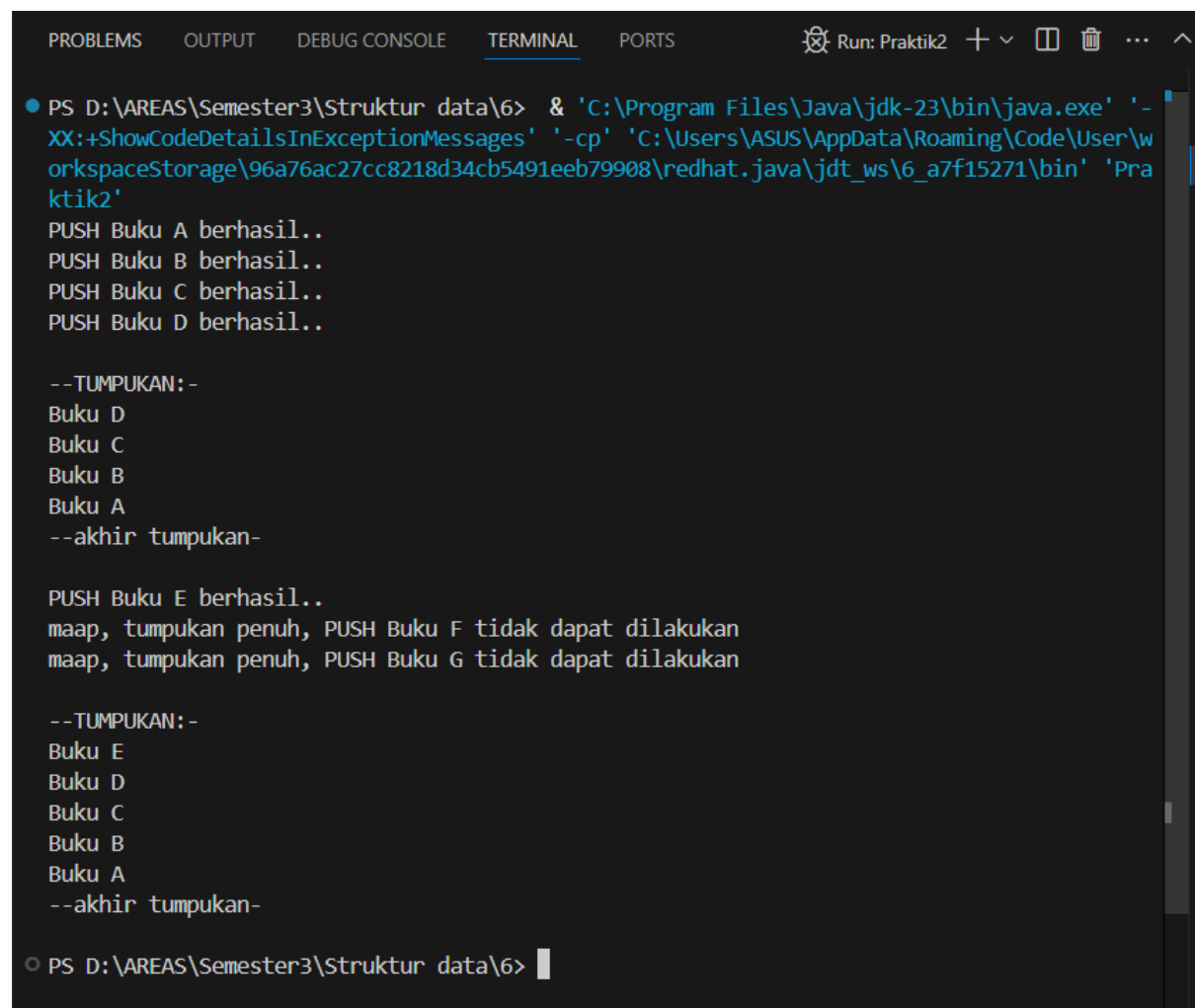
    public static void main(String[] args) {
        String tumpukan[] = new String[N];

        // Praktik 1
        PUSH(tumpukan, "Buku A");
        PUSH(tumpukan, "Buku B");
        PUSH(tumpukan, "Buku C");
        PUSH(tumpukan, "Buku D");
        lihatTumpukan(tumpukan);
    }
}

```

Pembahasan : Program diatas adalah program Java yang mensimulasikan struktur data tumpukan (stack) menggunakan array. Di dalam program ini, terdapat sebuah array bernama tumpukan yang ukurannya ditentukan oleh variabel N dengan nilai 5. Ada juga variabel atas yang berfungsi sebagai penunjuk posisi data teratas dalam tumpukan, diinisialisasi dengan nilai -1 yang menandakan tumpukan masih kosong. Program ini memiliki method PUSH yang digunakan untuk menambahkan data ke dalam tumpukan. Pada method main, program memanggil method PUSH sebanyak empat kali untuk memasukkan "Buku A", "Buku B", "Buku C", dan "Buku D". Setiap kali method PUSH dipanggil, ia akan menaikkan nilai variabel atas terlebih dahulu, lalu memasukkan data baru ke tumpukan pada indeks yang ditunjuk oleh atas. Terakhir, program memanggil method lihatTumpukan untuk menampilkan isi dari tumpukan tersebut.

Praktik 2



```
PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik2'
PUSH Buku A berhasil..
PUSH Buku B berhasil..
PUSH Buku C berhasil..
PUSH Buku D berhasil..

--TUMPUKAN:-
Buku D
Buku C
Buku B
Buku A
--akhir tumpukan-

PUSH Buku E berhasil..
maap, tumpukan penuh, PUSH Buku F tidak dapat dilakukan
maap, tumpukan penuh, PUSH Buku G tidak dapat dilakukan

--TUMPUKAN:-
Buku E
Buku D
Buku C
Buku B
Buku A
--akhir tumpukan-

PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik2 {
    public static int N = 5;
    public static int atas = -1;

    public static void PUSH(String tumpukan[], String data
) {
        if (atas == N - 1) { //jika tumpukan penuh
            System.out.println(
"maap, tumpukan penuh, PUSH " + data +
" tidak dapat dilakukan");
        } else { //jika tumpukan tidak penuh
            atas = atas + 1;
            tumpukan[atas] = data;
            System.out.println("PUSH " + data +
" berhasil..");
        }
    }

    public static String POP(String tumpukan[]) {
        String hasil;
        if (atas < 0) { //jika tumpukan kosong
            hasil = "TUMPUKAN KOSONG, POP GAGAL DILAKUKAN"
;
        } else { //jika tumpukan tidak kosong
            hasil = tumpukan[atas];
            atas = atas - 1;
        }
        return (hasil);
    }

    public static void lihatTumpukan(String tumpukan[]) {
        System.out.println("");
        System.out.println("--TUMPUKAN:-");
        for (int i = atas; i >= 0; i--) {
            System.out.println(tumpukan[i]);
        }
        System.out.println("--akhir tumpukan-");
        System.out.println("");
    }

    public static void main(String[] args) {
        String tumpukan[] = new String[N];

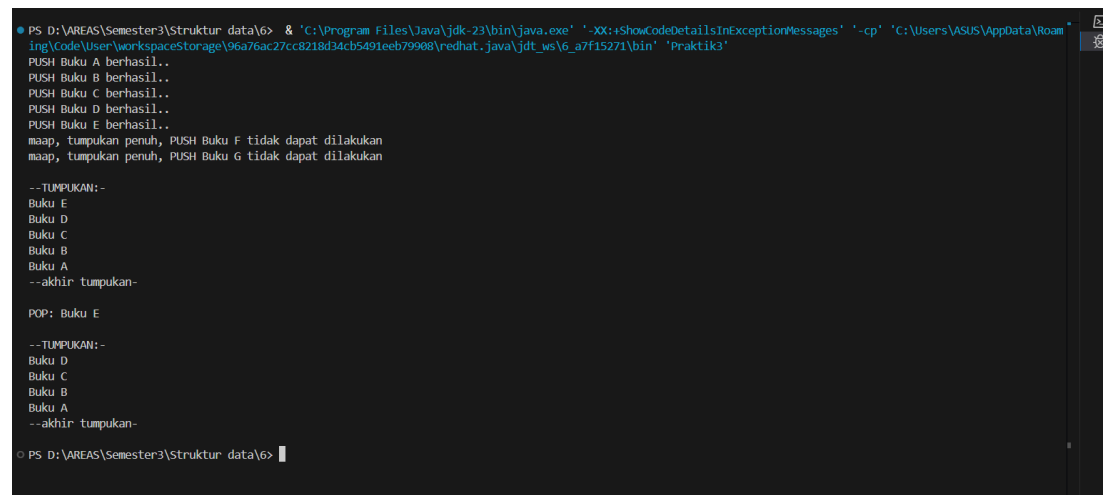
        // Praktik 1
        PUSH(tumpukan, "Buku A");
        PUSH(tumpukan, "Buku B");
        PUSH(tumpukan, "Buku C");
        PUSH(tumpukan, "Buku D");
        lihatTumpukan(tumpukan);

        // Praktik 2
        PUSH(tumpukan, "Buku E");
        PUSH(tumpukan, "Buku F");
        PUSH(tumpukan, "Buku G");
        lihatTumpukan(tumpukan);
    }
}

```

Pembahasan : Program kedua ini adalah kelanjutan dari program sebelumnya, yang masih menggunakan array tumpukan dengan ukuran N sebesar 5 dan variabel atas sebagai penunjuk. Setelah empat data dimasukkan pada Praktik 1, program ini kembali memanggil method PUSH untuk menambahkan "Buku E", "Buku F", dan "Buku G". Method PUSH memiliki sebuah kondisi if ($atas == N - 1$) untuk memeriksa apakah tumpukan sudah penuh. Karena N adalah 5, maka tumpukan penuh jika atas bernilai 4. Saat PUSH "Buku E" dijalankan, atas menjadi 4 dan tumpukan menjadi penuh. Akibatnya, ketika program mencoba menjalankan PUSH untuk "Buku F" dan "Buku G", kondisi if ($atas == N - 1$) akan terpenuhi, sehingga program akan menampilkan pesan bahwa tumpukan sudah penuh dan data tersebut tidak dapat dimasukkan.

Praktik 3



```
PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roam\
ing\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik3'
PUSH Buku A berhasil..
PUSH Buku B berhasil..
PUSH Buku C berhasil..
PUSH Buku D berhasil..
PUSH Buku E berhasil..
maap, tumpukan penuh, PUSH Buku F tidak dapat dilakukan
maap, tumpukan penuh, PUSH Buku G tidak dapat dilakukan

--TUMPUKAN:--
Buku E
Buku D
Buku C
Buku B
Buku A
--akhir tumpukan--

POP: Buku E

--TUMPUKAN:--
Buku D
Buku C
Buku B
Buku A
--akhir tumpukan--

PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik3 {
    public static int N = 5;
    public static int atas = -1;

    public static void PUSH(String tumpukan[], String data) {
        if (atas == N - 1) { //jika tumpukan penuh
            System.out.println("maap, tumpukan penuh, PUSH " + data + " tidak dapat dilakukan");
        } else { //jika tumpukan tidak penuh
            atas = atas + 1;
            tumpukan[atas] = data;
            System.out.println("PUSH " + data + " berhasil..");
        }
    }

    public static String POP(String tumpukan[]) {
        String hasil;
        if (atas < 0) { //jika tumpukan kosong
            hasil = "TUMPUKAN KOSONG, POP GAGAL DILAKUKAN";
        } else { //jika tumpukan tidak kosong
            hasil = tumpukan[atas];
            atas = atas - 1;
        }
        return (hasil);
    }

    public static void lihatTumpukan(String tumpukan[]) {
        System.out.println("");
        System.out.println("--TUMPUKAN--");
        for (int i = atas; i >= 0; i--) {
            System.out.println(tumpukan[i]);
        }
        System.out.println("--akhir tumpukan--");
        System.out.println("");
    }

    public static void main(String[] args) {
        String tumpukan[] = new String[N];

        // Praktik 1
        PUSH(tumpukan, "Buku A");
        PUSH(tumpukan, "Buku B");
        PUSH(tumpukan, "Buku C");
        PUSH(tumpukan, "Buku D");

        // Praktik 2
        PUSH(tumpukan, "Buku E");
        PUSH(tumpukan, "Buku F");
        PUSH(tumpukan, "Buku G");
        lihatTumpukan(tumpukan);

        // Praktik 3
        System.out.println("POP: " + POP(tumpukan));
        lihatTumpukan(tumpukan);
    }
}

```

Pembahasan : Pada program ketiga, diperkenalkan sebuah method baru yaitu POP. Method POP ini berfungsi untuk mengambil dan menghapus data teratas dari tumpukan. Berbeda dengan PUSH yang menambahkan data, POP justru mengurangi data. Di dalam method main, setelah tumpukan terisi penuh, program memanggil `System.out.println("POP: " + POP(tumpukan))`. Method POP akan mengambil nilai dari tumpukan pada indeks yang ditunjuk oleh variabel atas, yaitu "Buku E". Setelah itu, method POP akan mengurangi nilai dari variabel atas sebanyak satu. Dengan berkurangnya nilai atas, data "Buku E" secara efektif tidak lagi menjadi bagian dari tumpukan. Hasil dari method POP kemudian ditampilkan ke layar, dan `lihatTumpukan` dipanggil lagi untuk menunjukkan isi tumpukan yang sudah berkurang.

Praktik 4

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik4'
PUSH Buku A berhasil..
PUSH Buku B berhasil..
PUSH Buku C berhasil..
PUSH Buku D berhasil..
PUSH Buku E berhasil..
POP: Buku E

--TUMPUKAN:-
Buku D
Buku C
Buku B
Buku A
--akhir tumpukan-

POP: Buku D
POP: Buku C

--TUMPUKAN:-
Buku B
Buku A
--akhir tumpukan-

○ PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik4 {
    public static int N = 5;
    public static int atas = -1;

    public static void PUSH(String tumpukan[], String data
) {
        if (atas == N - 1) { //jika tumpukan penuh
            System.out.println(
"maap, tumpukan penuh, PUSH " + data +
" tidak dapat dilakukan");
        } else { //jika tumpukan tidak penuh
            atas = atas + 1;
            tumpukan[atas] = data;
            System.out.println("PUSH " + data +
" berhasil..");
        }
    }

    public static String POP(String tumpukan[]) {
        String hasil;
        if (atas < 0) { //jika tumpukan kosong
            hasil = "TUMPUKAN KOSONG, POP GAGAL DILAKUKAN"
;
        } else { //jika tumpukan tidak kosong
            hasil = tumpukan[atas];
            atas = atas - 1;
        }
        return (hasil);
    }

    public static void lihatTumpukan(String tumpukan[]) {
        System.out.println("");
        System.out.println("--TUMPUKAN:-");
        for (int i = atas; i >= 0; i--) {
            System.out.println(tumpukan[i]);
        }
        System.out.println("--akhir tumpukan-");
        System.out.println("");
    }

    public static void main(String[] args) {
        String tumpukan[] = new String[N];

        // Praktik 1 & 2
        PUSH(tumpukan, "Buku A");
        PUSH(tumpukan, "Buku B");
        PUSH(tumpukan, "Buku C");
        PUSH(tumpukan, "Buku D");
        PUSH(tumpukan, "Buku E");

        // Praktik 3
        System.out.println("POP: " + POP(tumpukan));
        lihatTumpukan(tumpukan);

        // Praktik 4
        System.out.println("POP: " + POP(tumpukan));
        System.out.println("POP: " + POP(tumpukan));
        lihatTumpukan(tumpukan);
    }
}

```

Pembahasan : Program keempat ini melanjutkan penggunaan method POP pada tumpukan yang sama. Setelah sebelumnya satu data diambil menggunakan POP pada Praktik 3, di program ini method POP dipanggil lagi sebanyak dua kali. Panggilan POP yang pertama akan mengambil data teratas saat itu, yaitu "Buku D", dan mengurangi nilai variabel atas. Panggilan POP yang kedua akan mengambil data teratas berikutnya, yaitu "Buku C", dan kembali mengurangi nilai variabel atas. Setiap kali POP dipanggil, nilai yang diambil akan ditampilkan di layar. Setelah dua kali pemanggilan POP, method lihatTumpukan dieksekusi untuk memperlihatkan keadaan akhir dari tumpukan, yang sekarang hanya berisi "Buku B" dan "Buku A".

Praktik 5

```
TERMINAL  ...  Run: Praktik5  +  v  [ ]  [X]  ...  ^

PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program
Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsI
nExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roami
ng\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491
eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik5'
PUSH Buku A berhasil..
PUSH Buku B berhasil..
PUSH Buku C berhasil..
PUSH Buku D berhasil..
PUSH Buku E berhasil..
POP: Buku B
POP: Buku A
POP: TUMPUKAN KOSONG, POP GAGAL DILAKUKAN
POP: TUMPUKAN KOSONG, POP GAGAL DILAKUKAN

--TUMPUKAN:-
--akhir tumpukan-

PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik5 {
    public static int N = 5;
    public static int atas = -1;

    public static void PUSH(String tumpukan[], String data) {
        if (atas == N - 1) { //jika tumpukan penuh
            System.out.println("maap, tumpukan penuh, PUSH " + data + " tidak dapat dilakukan");
        } else { //jika tumpukan tidak penuh
            atas = atas + 1;
            tumpukan[atas] = data;
            System.out.println("PUSH " + data + " berhasil..");
        }
    }

    public static String POP(String tumpukan[]) {
        String hasil;
        if (atas < 0) { //jika tumpukan kosong
            hasil = "TUMPUKAN KOSONG, POP GAGAL DILAKUKAN";
        } else { //jika tumpukan tidak kosong
            hasil = tumpukan[atas];
            atas = atas - 1;
        }
        return (hasil);
    }

    public static void lihatTumpukan(String tumpukan[]) {
        System.out.println("");
        System.out.println("--TUMPUKAN--");
        for (int i = atas; i >= 0; i--) {
            System.out.println(tumpukan[i]);
        }
        System.out.println("--akhir tumpukan--");
        System.out.println("");
    }

    public static void main(String[] args) {
        String tumpukan[] = new String[N];

        // Praktik 1 & 2
        PUSH(tumpukan, "Buku A");
        PUSH(tumpukan, "Buku B");
        PUSH(tumpukan, "Buku C");
        PUSH(tumpukan, "Buku D");
        PUSH(tumpukan, "Buku E");

        // Praktik 3
        POP(tumpukan);

        // Praktik 4
        POP(tumpukan);
        POP(tumpukan);

        // Praktik 5
        System.out.println("POP: " + POP(tumpukan));
        System.out.println("POP: " + POP(tumpukan));
        System.out.println("POP: " + POP(tumpukan));
        System.out.println("POP: " + POP(tumpukan));
        lihatTumpukan(tumpukan);
    }
}

```

Pembahasan : Program kelima ini adalah puncak dari operasi POP pada tumpukan. Setelah tumpukan diisi penuh lalu isinya dikurangi tiga kali melalui method POP di praktik-praktik sebelumnya, program ini mencoba memanggil POP berulang kali hingga tumpukan kosong dan bahkan lebih. Panggilan POP pertama dan kedua berhasil mengambil "Buku B" dan "Buku A", membuat tumpukan menjadi kosong (variabel atas menjadi -1). Namun, program terus memanggil POP setelah tumpukan kosong. Di dalam method POP, terdapat kondisi if (atas < 0) yang akan terpenuhi jika tumpukan kosong. Ketika kondisi ini terpenuhi, method POP akan mengembalikan pesan "TUMPUKAN KOSONG, POP GAGAL DILAKUKAN". Pesan inilah yang akan ditampilkan untuk panggilan POP ketiga dan seterusnya pada praktik ini.

Praktik 6

```
TERMINAL  ...  Run: Praktik6  +  -  [ ]  [ ]  ...

● PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program
Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsI
nExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roami
ng\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491
eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik6'

ENQUEUE Mobil A berhasil..
ENQUEUE Mobil B berhasil..
ENQUEUE Mobil C berhasil..

ISI ANTRIAN : (depan) Mobil A Mobil B Mobil C (belakan
g)

○ PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik6 {
    public static int N = 5;
    public static int belakang = -1;

    public static void ENQUEUE(String antrian[], String data) {
        if (belakang == N - 1) { //jika antrian penuh
            System.out.println("maap, antrian penuh, ENQUEUE " + data + " tidak dapat dilakukan");
        } else { //jika antrian tidak penuh
            belakang = belakang + 1;
            antrian[belakang] = data;
            System.out.println("ENQUEUE " + data + " berhasil..");
        }
    }

    public static String DEQUEUE(String antrian[]) {
        String hasil;
        if (belakang < 0) { //jika antrian kosong
            hasil = "ANTRIAN KOSONG, DEQUEUE GAGAL DILAKUKAN";
        } else { //jika antrian tidak kosong
            hasil = antrian[0];
            //---menggeser data kedua dst, maju selangkah ke depan
            for (int i = 0; i <= belakang - 1; i++) {
                antrian[i] = antrian[i + 1];
            }
            belakang = belakang - 1;
        }
        return (hasil);
    }

    public static void lihatAntrian(String antrian[]) {
        System.out.println("");
        System.out.print("ISI ANTRIAN : (depan)");
        for (int i = 0; i <= belakang; i++) {
            System.out.print(" " + antrian[i]);
        }
        System.out.println(" (belakang)");
        System.out.println(" ");
    }

    public static void main(String[] args) {
        String antrian[] = new String[N];

        // Praktik 6
        ENQUEUE(antrian, "Mobil A");
        ENQUEUE(antrian, "Mobil B");
        ENQUEUE(antrian, "Mobil C");
        lihatAntrian(antrian);
    }
}

```

Pembahasan : Program keenam memperkenalkan konsep struktur data baru, yaitu antrian (queue), yang diimplementasikan menggunakan array antrian dengan ukuran N sebesar 5. Berbeda dengan tumpukan yang menggunakan variabel atas, antrian ini menggunakan variabel belakang sebagai penunjuk elemen terakhir, yang diinisialisasi dengan -1. Program ini memiliki method ENQUEUE untuk menambahkan data ke antrian. Pada method main, ENQUEUE dipanggil tiga kali untuk menambahkan "Mobil A", "Mobil B", dan "Mobil C". Cara kerja ENQUEUE mirip dengan PUSH, yaitu menaikkan nilai belakang lalu memasukkan data ke array antrian pada indeks belakang. Setelah itu, method lihatAntrian dipanggil untuk menampilkan seluruh isi antrian dari depan ke belakang.

Praktik 7

```
● PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program  
Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsI  
nExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roami  
ng\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491  
eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik7'
```

```
ENQUEUE Mobil A berhasil..
```

```
ENQUEUE Mobil B berhasil..
```

```
ENQUEUE Mobil C berhasil..
```

```
ENQUEUE Mobil D berhasil..
```

```
ENQUEUE Mobil E berhasil..
```

```
maap, antrian penuh, ENQUEUE Mobil F tidak dapat dilak  
ukan
```

```
ISI ANTRIAN : (depan) Mobil A Mobil B Mobil C Mobil D  
Mobil E (belakang)
```

```
○ PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik7 {
    public static int N = 5;
    public static int belakang = -1;

    public static void ENQUEUE(String antrian[], String data) {
        if (belakang == N - 1) { //jika antrian penuh
            System.out.println("maap, antrian penuh, ENQUEUE " + data + " tidak dapat dilakukan");
        } else { //jika antrian tidak penuh
            belakang = belakang + 1;
            antrian[belakang] = data;
            System.out.println("ENQUEUE " + data + " berhasil..");
        }
    }

    public static String DEQUEUE(String antrian[]) {
        String hasil;
        if (belakang < 0) { //jika antrian kosong
            hasil = "ANTRIAN KOSONG, DEQUEUE GAGAL DILAKUKAN";
        } else { //jika antrian tidak kosong
            hasil = antrian[0];
            //---menggeser data kedua dst, maju selangkah ke depan
            for (int i = 0; i <= belakang - 1; i++) {
                antrian[i] = antrian[i + 1];
            }
            belakang = belakang - 1;
        }
        return (hasil);
    }

    public static void lihatAntrian(String antrian[]) {
        System.out.println("");
        System.out.print("ISI ANTRIAN : (depan)");
        for (int i = 0; i <= belakang; i++) {
            System.out.print(" " + antrian[i]);
        }
        System.out.println(" (belakang)");
        System.out.println(" ");
    }

    public static void main(String[] args) {
        String antrian[] = new String[N];

        // Praktik 6
        ENQUEUE(antrian, "Mobil A");
        ENQUEUE(antrian, "Mobil B");
        ENQUEUE(antrian, "Mobil C");

        // Praktik 7
        ENQUEUE(antrian, "Mobil D");
        ENQUEUE(antrian, "Mobil E");
        ENQUEUE(antrian, "Mobil F");
        lihatAntrian(antrian);
    }
}

```

Pembahasan : Program ketujuh adalah kelanjutan dari program antrian sebelumnya. Program ini masih menggunakan array antrian yang sama dengan ukuran N sebesar 5 dan variabel belakang sebagai penunjuk. Setelah tiga data dimasukkan pada Praktik 6, program ini kembali memanggil method ENQUEUE untuk menambahkan "Mobil D", "Mobil E", dan "Mobil F". Sama seperti PUSH, method ENQUEUE juga memiliki kondisi if (belakang == N - 1) untuk memeriksa apakah antrian sudah penuh. ENQUEUE untuk "Mobil D" dan "Mobil E" berhasil dieksekusi, membuat antrian penuh dengan belakang bernilai 4. Ketika program mencoba ENQUEUE "Mobil F", kondisi if (belakang == N - 1) terpenuhi, sehingga proses ENQUEUE gagal dan program menampilkan pesan bahwa antrian penuh.

Praktik 8

```
● PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik8'
```

```
ENQUEUE Mobil A berhasil..  
ENQUEUE Mobil B berhasil..  
ENQUEUE Mobil C berhasil..  
ENQUEUE Mobil D berhasil..  
ENQUEUE Mobil E berhasil..  
deQueue: Mobil A
```

```
ISI ANTRIAN : (depan) Mobil B Mobil C Mobil D Mobil E  
(belakang)
```

```
○ PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktik8 {
    public static int N = 5;
    public static int belakang = -1;

    public static void ENQUEUE(String antrian[], String data) {
        if (belakang == N - 1) { //jika antrian penuh
            System.out.println("maap, antrian penuh, ENQUEUE " + data + " tidak dapat dilakukan");
        } else { //jika antrian tidak penuh
            belakang = belakang + 1;
            antrian[belakang] = data;
            System.out.println("ENQUEUE " + data + " berhasil..");
        }
    }

    public static String DEQUEUE(String antrian[]) {
        String hasil;
        if (belakang < 0) { //jika antrian kosong
            hasil = "ANTRIAN KOSONG, DEQUEUE GAGAL DILAKUKAN";
        } else { //jika antrian tidak kosong
            hasil = antrian[0];
            //---menggeser data kedua dst, maju selangkah ke depan
            for (int i = 0; i <= belakang - 1; i++) {
                antrian[i] = antrian[i + 1];
            }
            belakang = belakang - 1;
        }
        return (hasil);
    }

    public static void lihatAntrian(String antrian[]) {
        System.out.println("");
        System.out.print("ISI ANTRIAN : (depan)");
        for (int i = 0; i <= belakang; i++) {
            System.out.print(" " + antrian[i]);
        }
        System.out.println(" (belakang)");
        System.out.println(" ");
    }

    public static void main(String[] args) {
        String antrian[] = new String[N];

        // Praktik 6 & 7
        ENQUEUE(antrian, "Mobil A");
        ENQUEUE(antrian, "Mobil B");
        ENQUEUE(antrian, "Mobil C");
        ENQUEUE(antrian, "Mobil D");
        ENQUEUE(antrian, "Mobil E");

        // Praktik 8
        System.out.println("deQueue: " + DEQUEUE(antrian));
        lihatAntrian(antrian);
    }
}

```

Pembahasan : Pada program kedelapan, diperkenalkan method DEQUEUE yang berfungsi untuk mengambil dan menghapus data terdepan dari antrian. Ini adalah perbedaan utama antara antrian (FIFO - First-In, First-Out) dan tumpukan (LIFO - Last-In, First-Out). Di dalam method main, setelah antrian terisi penuh, method DEQUEUE dipanggil satu kali. Method DEQUEUE akan mengambil elemen pertama dari antrian (indeks ke-0), yaitu "Mobil A". Setelah mengambilnya, DEQUEUE melakukan perulangan for untuk menggeser semua elemen lain di antrian satu langkah ke depan. Akhirnya, nilai variabel belakang dikurangi satu. Hasil dari DEQUEUE ditampilkan ke layar, dan lihatAntrian dipanggil untuk menunjukkan isi antrian yang sudah bergeser.

Praktik 9

```
TERMINAL  ...  Run: Praktik9  +  v  [ ]  [X]  ...  ↗

● PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program
Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsI
nExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roami
ng\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491
eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik9'

ENQUEUE Mobil A berhasil..
ENQUEUE Mobil B berhasil..
ENQUEUE Mobil C berhasil..
ENQUEUE Mobil D berhasil..
ENQUEUE Mobil E berhasil..
deQueue: Mobil B
deQueue: Mobil C
deQueue: Mobil D

ISI ANTRIAN : (depan) Mobil E (belakang)

○ PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class PraktiK9 {
    public static int N = 5;
    public static int belakang = -1;

    public static void ENQUEUE(String antrian[], String data) {
        if (belakang == N - 1) { //jika antrian penuh
            System.out.println("maap, antrian penuh, ENQUEUE " + data + " tidak dapat dilakukan");
        } else { //jika antrian tidak penuh
            belakang = belakang + 1;
            antrian[belakang] = data;
            System.out.println("ENQUEUE " + data + " berhasil..");
        }
    }

    public static String DEQUEUE(String antrian[]) {
        String hasil;
        if (belakang < 0) { //jika antrian kosong
            hasil = "ANTRIAN KOSONG, DEQUEUE GAGAL DILAKUKAN";
        } else { //jika antrian tidak kosong
            hasil = antrian[0];
            //---menggeser data kedua dst, maju selangkah ke depan
            for (int i = 0; i <= belakang - 1; i++) {
                antrian[i] = antrian[i + 1];
            }
            belakang = belakang - 1;
        }
        return (hasil);
    }

    public static void lihatAntrian(String antrian[]) {
        System.out.println("");
        System.out.print("ISI ANTRIAN : (depan)");
        for (int i = 0; i <= belakang; i++) {
            System.out.print(" " + antrian[i]);
        }
        System.out.println(" (belakang)");
        System.out.println(" ");
    }

    public static void main(String[] args) {
        String antrian[] = new String[N];

        // PraktiK 6 & 7
        ENQUEUE(antrian, "Mobil A");
        ENQUEUE(antrian, "Mobil B");
        ENQUEUE(antrian, "Mobil C");
        ENQUEUE(antrian, "Mobil D");
        ENQUEUE(antrian, "Mobil E");

        // PraktiK 8
        DEQUEUE(antrian);

        // PraktiK 9
        System.out.println("deQueue: " + DEQUEUE(antrian));
        System.out.println("deQueue: " + DEQUEUE(antrian));
        System.out.println("deQueue: " + DEQUEUE(antrian));
        lihatAntrian(antrian);
    }
}

```

Pembahasan : Program kesembilan ini melanjutkan operasi DEQUEUE pada antrian. Setelah satu elemen dikeluarkan pada PraktiK 8, program ini memanggil method DEQUEUE sebanyak tiga kali lagi. Panggilan DEQUEUE pertama akan mengambil "Mobil B", panggilan kedua mengambil "Mobil C", dan panggilan ketiga mengambil "Mobil D". Setiap kali DEQUEUE dipanggil, elemen terdepan diambil, lalu semua elemen sisanya digeser maju menggunakan perulangan for, dan nilai variabel belakang dikurangi. Nilai yang di-DEQUEUE akan ditampilkan ke layar. Setelah tiga panggilan tersebut, lihatAntrian dieksekusi untuk menunjukkan keadaan antrian yang kini hanya tersisa satu elemen.

Praktik 10

```
TERMINAL  ...  Run: Praktik10  +  v  [ ]  [ ]  ...

● PS D:\AREAS\Semester3\Struktur data\6> & 'C:\Program
Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsI
nExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roami
ng\Code\User\workspaceStorage\96a76ac27cc8218d34cb5491
eeb79908\redhat.java\jdt_ws\6_a7f15271\bin' 'Praktik10
'

ENQUEUE Mobil A berhasil..
ENQUEUE Mobil B berhasil..
ENQUEUE Mobil C berhasil..
ENQUEUE Mobil D berhasil..
ENQUEUE Mobil E berhasil..
deQueue: Mobil E
deQueue: ANTRIAN KOSONG, DEQUEUE GAGAL DILAKUKAN

ISI ANTRIAN : (depan) (belakang)

○ PS D:\AREAS\Semester3\Struktur data\6>
```

```

public class Praktikum10 {
    public static int N = 5;
    public static int belakang = -1;

    public static void ENQUEUE(String antrian[], String data) {
        if (belakang == N - 1) { //jika antrian penuh
            System.out.println("maap, antrian penuh, ENQUEUE " + data + " tidak dapat dilakukan");
        } else { //jika antrian tidak penuh
            belakang = belakang + 1;
            antrian[belakang] = data;
            System.out.println("ENQUEUE " + data + " berhasil..");
        }
    }

    public static String DEQUEUE(String antrian[]) {
        String hasil;
        if (belakang < 0) { //jika antrian kosong
            hasil = "ANTRIAN KOSONG, DEQUEUE GAGAL DILAKUKAN";
        } else { //jika antrian tidak kosong
            hasil = antrian[0];
            //---menggeser data kedua dst, maju selangkah ke depan
            for (int i = 0; i <= belakang - 1; i++) {
                antrian[i] = antrian[i + 1];
            }
            belakang = belakang - 1;
        }
        return (hasil);
    }

    public static void lihatAntrian(String antrian[]) {
        System.out.println("");
        System.out.print("ISI ANTRIAN : (depan)");
        for (int i = 0; i <= belakang; i++) {
            System.out.print(" " + antrian[i]);
        }
        System.out.println(" (belakang)");
        System.out.println(" ");
    }

    public static void main(String[] args) {
        String antrian[] = new String[N];

        // Praktikum 6 & 7
        ENQUEUE(antrian, "Mobil A");
        ENQUEUE(antrian, "Mobil B");
        ENQUEUE(antrian, "Mobil C");
        ENQUEUE(antrian, "Mobil D");
        ENQUEUE(antrian, "Mobil E");

        // Praktikum 8
        DEQUEUE(antrian);

        // Praktikum 9
        DEQUEUE(antrian);
        DEQUEUE(antrian);
        DEQUEUE(antrian);

        // Praktikum 10
        System.out.println("deQueue: " + DEQUEUE(antrian));
        System.out.println("deQueue: " + DEQUEUE(antrian));
        lihatAntrian(antrian);
    }
}

```

Pembahasan : Program kesepuluh adalah tahap akhir dari operasi DEQUEUE pada antrian. Setelah antrian diisi penuh dan isinya dikeluarkan empat kali pada praktik sebelumnya, program ini mencoba memanggil DEQUEUE lagi sebanyak dua kali. Panggilan DEQUEUE yang pertama berhasil mengambil elemen terakhir yang tersisa, yaitu "Mobil E", yang membuat antrian menjadi kosong (variabel belakang menjadi -1). Ketika program mencoba memanggil DEQUEUE untuk kedua kalinya, kondisi if (belakang < 0) di dalam method DEQUEUE akan terpenuhi. Akibatnya, method DEQUEUE akan mengembalikan pesan "ANTRIAN KOSONG, DEQUEUE GAGAL DILAKUKAN", yang kemudian ditampilkan ke layar.

LATIHAN

Latihan 1

TUGAS

Tugas 1

Pembahasan :

D. KESIMPULAN

Dari keseluruhan praktikum, dapat disimpulkan bahwa struktur data tumpukan (stack) dan antrian (queue) dapat diimplementasikan menggunakan array. Tumpukan bekerja dengan prinsip LIFO (Last-In, First-Out), di mana data yang terakhir dimasukkan ('PUSH') adalah data yang pertama kali dikeluarkan ('POP'). Operasi ini dikelola oleh satu penunjuk, yaitu 'atas'. Sebaliknya, antrian bekerja dengan prinsip FIFO (First-In, First-Out), di mana data yang pertama kali dimasukkan ('ENQUEUE') adalah data yang pertama kali dikeluarkan ('DEQUEUE'). Implementasi antrian dengan array memerlukan proses penggeseran elemen setiap kali operasi 'DEQUEUE' dilakukan agar elemen terdepan selalu berada di indeks 0. Kedua struktur data ini memiliki batasan kapasitas yang ditentukan oleh ukuran array ('N'), dan program harus memiliki mekanisme untuk menangani kondisi saat struktur data penuh atau kosong untuk menghindari error.

E. LAMPIRAN